



PROGRAMAÇÃO PARA A INTERNET 2 (PROGWEB 2)

Caderno de Roteiros Práticos da Unidade Curricular

Curso Superior de Tecnologia em Sistemas para a Internet — IFSC Garopaba

Sumário

1	Visão Geral da Unidade Curricular	3
2	Roteiro Prático 01 — Retomada e Transição Parte 1: Bootstrap 5 & Layout Responsivo Estático	5
2.1	Introdução, Site Oficial e Inclusão do Bootstrap 5	5
2.2	Detalhamento Didático dos Exemplos de Bootstrap 5 (Exemplos 1 a 5)	6
2.3	Guia de Apoio: "Para Explorar Mais no Bootstrap 5"	6
2.4	Tarefa Prática de Laboratório (Construção do Layout Estático)	7
2.5	Checklist Final da Aula 1	7
3	Roteiro Prático 02 — Retomada e Transição Parte 2: jQuery, Manipulação do DOM & LocalStorage	8
3.1	Introdução, Sites Oficiais e Organização em Arquivo Externo app.js	8
3.2	Conceitos Fundamentais: O que é o DOM (Document Object Model)?	9
3.3	Sintaxe Básica do jQuery & O Comando \$(document).ready()	9
3.4	Metodologia Pedagógica Incremental: A Mesma Aplicação Base, Recursos Crescentes	10
3.5	Detalhamento Didático da Estrutura HTML Base Unificada & Exemplos 6 a 11	10
3.6	Guia de Apoio: "Para Explorar Mais no jQuery"	12
3.7	Parte 2 — Tarefa Prática: Programação Dinâmica em Arquivo Externo app.js	12
3.8	Parte 3 — Desafio Extra: Filtro Dinâmico em Tempo Real	13
3.9	Checklist Final da Aula 2	13
4	Roteiro Prático 03 — Atividade Prática Autônoma: Calculadora de Pedidos & Carrinho de Compras	14
4.1	Introdução e Contextualização da Atividade	14
4.2	Fundamentação Teórica: Como Ler e Calcular Valores de Campos com jQuery	15
4.2.1	1. Leitura e Conversão de Texto para Número (parseFloat e parseInt)	15
4.2.2	2. Soma de Múltiplos Checkboxes Marcados com .each()	15
4.2.3	3. Guia de Eventos do Navegador (input, change e click)	16



4.2.4	4. Formatação Financeira em Real (R\$)	16
4.3	Dicas de Estruturação Visual & Componentes do Bootstrap 5	16
4.4	Roteiro Guiado Passo a Passo (Roteiro 3)	17
4.5	Checklist de Conclusão e Orientações de Entrega	19
5	Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas	20
5.1	Introdução e Contextualização do Projeto	20
5.2	Estrutura Arquitetural de Pastas	21
5.3	Fundamentação Teórica: Como Funcionam os ES6 Modules	21
5.3.1	1. Declaração no HTML (type="module")	21
5.3.2	2. Exportação e Importação (export e import)	22
5.4	Passo a Passo Guiado do Roteiro	23
5.5	Checklist de Conclusão e Critérios Avaliativos	32



1 Visão Geral da Unidade Curricular

A Unidade Curricular de **Programação para a Internet II** tem como objetivo aprofundar os conhecimentos adquiridos na disciplina antecedente (Programação para a Internet I), evoluindo o desenvolvimento frontend básico em HTML5, CSS3 e JavaScript Vanilla para a construção de **Aplicações Web Modernas, Reativas, Modulares e Integradas a Serviços Web RESTful com aplicação de extensão comunitária**.

Ao longo do semestre, em encontros semanais de **3h/aula em laboratório**, os estudantes irão desenvolver competências avançadas em arquitetura de código client-side, modularização com ES6 Modules, manipulação avançada de estado no cliente, consumo de APIs assíncronas, roteamento no cliente (Single Page Applications — SPAs), introdução a ferramentas modernas de build (Vite) e desenvolvimento de um Projeto Prático Integrador de Extensão.

Estrutura do Semestre

O semestre está organizado em 5 módulos estruturantes de aprendizagem prática:

1. **Retomada e Diagnóstico (Revisão de PI-I)**

Revisão prática de manipulação do DOM, elementos visuais (Bootstrap 5), jQuery, assincronismo com `async/await`, Fetch API e armazenamento com `localStorage`.

2. **Arquitetura Modular e ES6 Modules**

Organização profissional de código frontend, separação de responsabilidades em serviços, componentes e manipuladores de eventos.

3. **Gestão Avançada de Estado e Formulários Reativos**

Persistência no navegador, validações avançadas no cliente, tratamento rigoroso de erros e experiência do usuário (UX).

4. **Consumo de APIs RESTful e Desenvolvimento de SPAs**

Construção de Single Page Applications com roteamento client-side sem recarregamento de página e integração com rotas de backend.

5. **Introdução a Ecossistemas Modernos e Projeto de Extensão**

Uso de bundlers (`Vite`), introdução à componentização reativa e entrega de um Projeto Prático Integrador de Extensão completo de final de semestre.

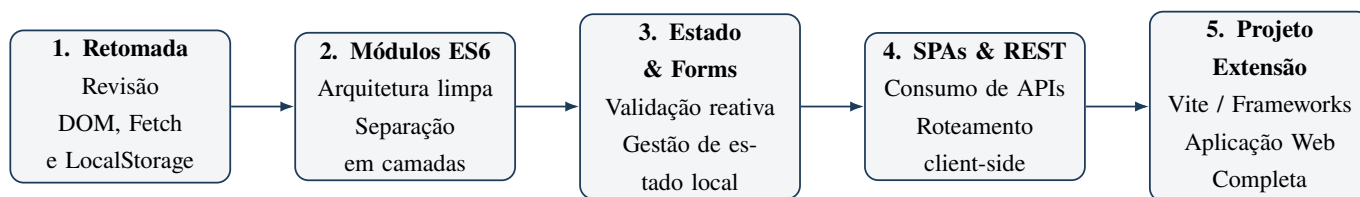


Figura 1: Pipeline de aprendizagem da UC Programação para a Internet II.

Competências Desenvolvidas

Ao final da disciplina, o estudante deverá ser capaz de:

- Construir interfaces web altamente dinâmicas e responsivas;
- Estruturar aplicações JavaScript utilizando arquitetura modular em ES6 Modules;
- Consumir e manipular dados de serviços Web RESTful usando `fetch` e `async/await`;
- Gerenciar estado local e persistência no cliente via Web Storage API;
- Implementar padrões de projeto de Single Page Application (SPA);
- Utilizar ferramentas modernas de ambiente de desenvolvimento web (Node.js, npm, Vite);
- Desenvolver um projeto prático integrador de extensão atendendo demandas reais da comunidade;
- Publicar e versionar aplicações web profissionais no GitHub e GitHub Pages.

Metodologia de Avaliação

A avaliação é contínua e formativa, dividida em três eixos principais:

1. **Avaliação Teórica Única (A1 — 25%):** Prova teórica individual de verificação dos conceitos fundamentais de arquitetura modular, assincronismo, protocolo HTTP, consumo de APIs e SPAs.
2. **Avaliação Contínua de Aulas (A2 — 35%):** Conjunto de **Participação Ativa + Pontualidade de Entrega** dos roteiros práticos semanais desenvolvidos nos laboratórios.
3. **Trabalho Prático Integrador / Extensão (A3 — 40%):** Desenvolvendo uma aplicação web completa de extensão comunitária/acadêmica que integra todos os conteúdos ministrados na unidade curricular (Bootstrap 5, JS ES6 Modular, Fetch API, LocalStorage, Roteamento SPA e Vite), hospedada no GitHub Pages com defesa demonstrativa.



2 Roteiro Prático 01 — Retomada e Transição Parte 1: Bootstrap 5 & Layout Responsivo Estático

Objetivos

Objetivos da Aula (Parte 1 — Bootstrap 5)

Ao final desta aula o estudante será capaz de:

- Conectar a base de HTML5 semântico e CSS3 de **ProgWeb 1** com a evolução prática de **ProgWeb 2**;
- Compreender o alinhamento do semestre com o Projeto Pedagógico do Curso (**PPC CONSUP 30/2020 — 120h**);
- Localizar e obter a biblioteca no site oficial do **Bootstrap 5** (<https://getbootstrap.com>);
- Configurar o ambiente incluindo os arquivos de estilo CSS e Bootstrap Icons no <head> do documento HTML;
- Aplicar o sistema de **Grid Responsivo de 12 Colunas** do Bootstrap 5 (`.container`, `.row`, `.col-md-*`);
- Estilizar elementos visuais da interface (Exemplos 1 a 5: Grid, Formulários, Cards, Badges, Modais e Tabelas Zebradas);
- Estruturar a interface do **Layout Estático Completo da Aplicação Web** (Exemplo 5);
- Acompanhar o material através da apresentação de slides interativa da Parte 1 (<https://chameoandre.github.io/CST-Sistemas-para-internet-programacao-para-internet-2/retomada-de-atividades-pi2/parte-1-bootstrap/index.html>).

Introdução, Site Oficial e Inclusão do Bootstrap 5

Bem-vindos à primeira parte da retomada prática da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Neste primeiro encontro, focamos na construção da interface visual responsiva estática do nosso sistema utilizando o framework **Bootstrap 5**.

1. **Bootstrap 5 (Site Oficial & CDN):** No portal oficial <https://getbootstrap.com> e nas referências do W3Schools, obtemos a documentação de componentes e os links de CDN (jsDelivr) incluídos



via tag `<link rel="stylesheet">` na seção `<head>`;

2. **Bootstrap Icons:** Inclusão do pacote de ícones vetoriais oficial do Bootstrap para ilustrar botões, cartões e alertas da aplicação;
3. **Acompanhamento Interativo:** O trabalho é guiado pelos slides da Parte 1 (`parte-1-bootstrap/index.html`) e pela prática na pasta `exercicio-retomada/`.

Detalhamento Didático dos Exemplos de Bootstrap 5 (Exemplos 1 a 5)

Abaixo está o detalhamento dos componentes de layout desenvolvidos nesta etapa:

- **Exemplo 1 (Bootstrap 5 — Grid Responsivo & Containers):** *Desenvolvimento de layout responsivo flexível.* A classe `.container` centraliza o conteúdo, `.row` cria a linha de grade e `.col-12`, `.col-md-6`, `.col-lg-4` ajusta a largura para 100% no celular, 50% em tablets e 33.3% em desktops;
- **Exemplo 2 (Bootstrap 5 — Formulários & Inputs Estilizados):** *Construção de campos com visual moderno.* A classe `.mb-3` adiciona margem inferior, `.form-label` formata o rótulo e `.form-control` estiliza o campo com bordas arredondadas e efeito de foco azul;
- **Exemplo 3 (Bootstrap 5 — Cards, Botões & Alertas Visuais):** *Empacotamento de componentes de interface.* Cria um cartão com elevação visual (`.card .shadow`), exibe um alerta verde de confirmação (`.alert .alert-success`) e insere um botão estilizado (`.btn .btn-primary`);
- **Exemplo 4 (Bootstrap 5 — Tabelas Elegantes & Utilitários):** *Formatação de tabelas com múltiplos registros.* Aplica `.table` para espaçamento automático, `.table-hover` para destacar a linha sob o mouse, `.table-striped` para alternar as cores das linhas (zebrado visual) e `.badge` para exibir tags coloridas de status;
- **Exemplo 5 (Bootstrap 5 — Layout Estático Completo):** *Layout integrado de cadastro e listagem.* Combina Grid responsivo (`.row .col-lg-4` e `.col-lg-8`), Formulário no Card e Tabela Zebrada em uma única interface completa, pronta para receber a interatividade na Parte 2.

Guia de Apoio: "Para Explorar Mais no Bootstrap 5"

Para dar suporte aos alunos na customização do layout, o quadro abaixo resume opções adicionais de classes do Bootstrap 5:

**Opções Adicionais do Bootstrap 5 (Layout, Componentes e Utilitários)**

Código / Classe	Descrição e Efeito Visual
col-12 col-sm-6 col-lg-3	Ocupa 100% no celular, 50% em tablets pequenos e 25% em desktops wide.
form-select / form-control-lg	Aplica o estilo de menu suspenso ou aumenta o tamanho do campo input.
btn-outline-danger	Botão vazado vermelho que preenche ao passar o ponteiro do mouse.
shadow-sm / shadow-lg	Aplica sombra suave ou pronunciada para destacar cartões da página.
d-flex justify-content-between	Ativa o Flexbox e alinha itens nas extremidades da caixa.

Tarefa Prática de Laboratório (Construção do Layout Estático)**Atividade Prática – Layout Estático com Bootstrap 5**

Na pasta `exercicio-retomada/`, os estudantes deverão construir a estrutura estática:

- Inclusão dos Links no Head:** Abrir o arquivo `index.html` e incluir os links de CDN do Bootstrap 5 CSS e Bootstrap Icons;
- Montagem da Grade Responsiva:** Criar a estrutura do Grid dividindo a página entre a coluna de cadastro (`col-md-4`) e a coluna da tabela (`col-md-8`);
- Estilização de Formulários e Tabelas:** Aplicar as classes `.card`, `.form-control`, `.btn-primary` e `.table-striped` construindo o leiaute profissional completo (Exemplo 5).

Checklist Final da Aula 1**Verificação Técnica Parte 1**

Antes de finalizar o encontro, verifique se seu projeto atende aos critérios de aceitação:

As dependências do Bootstrap 5 CSS e Icons estão corretamente vinculadas no `<head>`;

A página ajusta seu layout de forma fluida ao alterar a largura da janela do navegador;

A interface estática da aplicação (Exemplo 5) está completamente montada e legível.



3 Roteiro Prático 02 — Retomada e Transição Parte 2: jQuery, Manipulação do DOM & LocalStorage

Objetivos

Objetivos da Aula (Parte 2 — jQuery & Dinâmica Frontend)

Ao final desta aula o estudante será capaz de:

- Retomar o layout estático construído no Bootstrap 5 e adicionar inteligência no lado do cliente com **jQuery**;
- Localizar e obter a biblioteca no portal oficial do **jQuery** (<https://jquery.com> e <https://code.jquery.com>);
- Incluir os scripts no final do <body> e organizar a lógica em arquivo JavaScript externo (app.js);
- Compreender o manipulador de inicialização segura `$(document).ready(function() {...});`;
- Selecionar elementos do DOM com `$(...)` e escutar eventos de formulário e clique (`.on('submit')`, `.click()`);
- Aplicar efeitos visuais e animações fluidas na interface (`.fadeIn()`, `.slideToggle()`, `.addClass()`);
- Inserir e remover elementos dinamicamente na árvore DOM com `.append()`, `.val()` e `.remove()`;
- Persistir e recuperar dados no navegador utilizando o **LocalStorage** (`setItem`, `getItem`, `JSON.stringify`, `JSON.parse`);
- Implementar filtro de busca dinâmico em tempo real na tabela;
- Acompanhar o material através da apresentação de slides interativa da Parte 2 (<https://chameoandre.github.io/CST-Sistemas-para-internet-programacao-para-internet-2/retomada-de-atividades-pi2/parte-2-jquery/index.html>).

Introdução, Sites Oficiais e Organização em Arquivo Externo app.js

Bem-vindos à segunda parte da retomada prática da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!



Nesta aula, transformamos a interface estática do Bootstrap 5 em um sistema ****dinâmico, interativo e funcional**** utilizando a biblioteca ****jQuery****.

1. **jQuery (Site Oficial & CDN):** No portal <https://jquery.com> e no servidor de CDN <https://code.jquery.com>, obtemos os links da versão minificada (`jquery-3.7.1.min.js`), incluída via tag `<script>` antes do fechamento do `</body>`;
2. **Organização Profissional em Arquivo Externo (`app.js`):** Adotamos como boa prática o desacoplamento do código JavaScript em um arquivo separado (`app.js`), inicializado com `$(document).ready()`;
3. **Acompanhamento Interativo:** O trabalho é guiado pelos slides da Parte 2 (`parte-2-jquery/index.html`) e pela prática na pasta `exercicio-retomada/`.

Conceitos Fundamentais: O que é o DOM (Document Object Model)?

Antes de manipularmos os elementos da página com o jQuery, é fundamental compreender o papel do **DOM (Document Object Model)**:

- **Significado da Sigla:** *Document Object Model* (Modelo de Objeto do Documento). É a representação em memória (estrutura em árvore) que o navegador cria ao ler o arquivo HTML, transformando cada tag em um objeto manipulável;
- **Responsabilidades Principais do DOM:**
 1. **Mapeamento Hierárquico:** Organiza a relação estrutural entre elementos pais e filhos (ex: `<form>` contendo `<input>`);
 2. **Interface de Interatividade:** Fornece comandos para ler valores, alterar textos, aplicar estilos CSS e escutar eventos do usuário (cliques e submissões);
- **Interação via JavaScript/jQuery:** O código JS consulta a árvore usando seletores como `$('#input-nome')`. A partir daí, executa operações dinâmicas: lê dados (`.val()`), altera textos (`.html()`), modifica cores (`.css()`) e reage a eventos (`.on('submit')`).

Sintaxe Básica do jQuery & O Comando `$(document).ready()`

Antes de manipularmos o formulário, precisamos compreender como as instruções são estruturadas e por que o método de inicialização é indispensável:

- **A Sintaxe Padrão `$(seletor).acao()`:**
 - `$`: É o atalho para invocar a biblioteca jQuery;
 - `seletor`: A string com o seletor CSS do elemento desejado (ex: `'#btn-salvar'`);



– `acao()`: O método que lê, modifica ou escuta o elemento (ex: `.val()`, `.html()`, `.on()`);

- **O que faz o `$(document).ready(function() {...})`:** É a função de **inicialização segura**. Ela garante que qualquer código JavaScript/jQuery só seja executado **após o navegador ter concluído a leitura do HTML e a montagem da árvore do DOM na memória RAM**;
- **Por que temos que utilizá-lo (A causa do erro):** Se o script rodar antes de o HTML ser lido, o elemento (ex: `#form-cadastro`) ainda não existirá na memória. O jQuery retornará um objeto nulo/vazio e qualquer tentativa de escutar um clique ou submissão falhará silenciosamente;
- **Sintaxe Abreviada Equivalente:** A comunidade também utiliza a forma curta moderna: `$(function() { /* código seguro */ });`.

Metodologia Pedagógica Incremental: A Mesma Aplicação Base, Recursos Crescentes

Para garantir que o aprendizado seja fluido e contínuo, adotamos a ****Metodologia Incremental****:

- **Estrutura Base Unificada:** Em vez de criar um código HTML novo do zero para cada exercício, mantemos a mesma página de ****Cadastro de Clientes**** criada na Parte 1 com Bootstrap 5;
- **Acúmulo de Superpoderes Passo a Passo:**
 1. **Exemplo 6 (Base):** Leitura de inputs com `.val()` e escrita em alerta com `.html()`;
 2. **Exemplo 7 (Feedback):** Remoção de `.d-none`, `.addClass('alert-success')` e piscar animado 2x com `.fadeOut().fadeIn()`;
 3. **Exemplo 8 (Estilos):** Troca dinâmica de cores de fundo e bordas via `.css({ ... })`;
 4. **Exemplo 9 (Efeitos):** Deslizamento de painel retrátil com `.slideToggle()` e `.slideDown()`;
 5. **Exemplo 10 (DOM Dinâmico):** Montagem de tabela dinâmica com `.append()` e remoção de linha pai com `.closest('tr').remove()`;
 6. **Exemplo 11 (Persistência):** Armazenamento permanente no navegador com `localStorage` e conversão `JSON.stringify/parse`.

Detalhamento Didático da Estrutura HTML Base Unificada & Exemplos 6 a 11

Para manter uma linha contínua de aprendizado sem perder a conexão entre os seletores, todos os exemplos da Parte 2 utilizam a ****mesma estrutura HTML base da aplicação de Cadastro de Clientes**** (Exemplo 5 do Bootstrap 5), incrementando-a passo a passo:

Estrutura HTML Base Unificada (Formulário, Alerta e Tabela):

```
<!-- Formulário de Cadastro (#form-cadastro) -->
<form id="form-cadastro" class="card p-3 mb-3">
```



```
<input type="text" id="input-nome"
      class="form-control mb-2" placeholder="Nome">
<input type="email" id="input-email"
      class="form-control mb-2" placeholder="E-mail">
<button type="submit" id="btn-salvar"
      class="btn btn-primary">Salvar Cliente</button>
</form>

<!-- Caixa de Alerta para Feedback Visual (#caixa-mensagem) -->
<div id="caixa-mensagem"
      class="alert alert-info d-none"></div>

<!-- Tabela com Corpo Dinâmico (#tabela-corpo) -->
<table class="table table-striped">
  <thead>
    <tr>
      <th>Nome</th>
      <th>E-mail</th>
      <th>Ação</th>
    </tr>
  </thead>
  <tbody id="tabela-corpo"></tbody>
</table>
```

Abaixo está a progressão funcional realizada sobre a mesma estrutura pedagógica:

- **Exemplo 6 (Passo 1 — Leitura & Escrita de Campos):** *Estrutura Base Inicial:* Captura a submissão do #form-cadastro, impede o recarregamento com `e.preventDefault()`, lê os valores de #input-nome e #input-email com `.val()` e escreve na caixa #caixa-mensagem com `.html()`;
- **Exemplo 7 (Passo 2 — Animações Visuais & Classes CSS):** *Adicionado em relação ao Exemplo 6:* Incrementa a #caixa-mensagem removendo a classe `.d-none`, aplicando `.addClass('alert-success')` e fazendo o alerta piscar/pulsar de forma altamente visível encadeando `.fadeOut(150).fadeIn(150).fadeOut(150)`;
- **Exemplo 8 (Passo 3 — Troca Dinâmica de Cores e Estilos CSS):** *Adicionado em relação ao Exemplo 7:* Altera o estilo diretamente via código com `.css()`, modificando a cor de fundo do formulário e as bordas/cores da caixa de mensagem de forma dinâmica;
- **Exemplo 9 (Passo 4 — Efeitos Visuais de Deslizamento):** *Adicionado em relação ao Exemplo 8:* Requer a criação do botão #btn-toggle-ajuda e do painel retrátil #painel-ajuda (`style="display: none;"`). Explora o efeito `.slideToggle(300)` para alternar o painel de instruções e `.slideDown(400)` para a mensagem surgir deslizando;
- **Exemplo 10 (Passo 5 — Manipulação Dinâmica do DOM & Tabela):** *Adicionado em relação ao Exemplo 9:* Insere a nova linha `<tr>` com os dados e o botão de exclusão na #tabela-corpo com



.append(). Ao clicar em Remover, exclui a linha pai com `$(this).closest('tr').fadeOut(300, function() { $(this).remove(); });`

- **Exemplo 11 (Passo 6 — Persistência no LocalStorage com JSON):** *Adicionado em relação ao Exemplo 10:* Salva o array de clientes no navegador com `localStorage.setItem('lista_clientes', JSON.stringify(clientes))` e restaura a tabela ao recarregar a página com `JSON.parse()`.

Guia de Apoio: "Para Explorar Mais no jQuery"

Para dar suporte aos alunos na programação dinâmica, o quadro abaixo resume métodos essenciais do jQuery:

Opções Adicionais de jQuery (Seletores, Efeitos e Manipulação do DOM)	
Método / Seletor	Descrição e Uso no Código
<code>\$('.minha-classe')</code>	Seleciona todos os elementos do DOM que possuem uma classe CSS específica.
<code>\$(el).slideDown() / .slideUp()</code>	Animação fluida de expandir para baixo ou recolher para cima.
<code>\$(el).attr('disabled', true)</code>	Lê ou altera atributos HTML de um elemento (ex: desativar botões).
<code>\$(el).prepend(html)</code>	Insere novo conteúdo HTML no início do elemento selecionado.
<code>\$(el).empty() / .remove()</code>	Limpa todo o conteúdo interno ou remove o próprio elemento da página.

Parte 2 — Tarefa Prática: Programação Dinâmica em Arquivo Externo app.js

Atividade Prática – Dinâmica com jQuery e LocalStorage

Na pasta `exercicio-retomada/`, os estudantes deverão tornar a aplicação 100% dinâmica:

1. **Inclusão do jQuery:** Adicionar a tag `<script src="https://code.jquery.com/jquery-3.7.1.min.js">` antes do `</body>`;
2. **Seleção e Eventos em app.js:** No arquivo externo `app.js`, capturar a submissão do formulário com `$('#formCadastro').on('submit');`
3. **Inserção Dinâmica no DOM:** Ler os campos com `$('#nome').val()` e adicionar a nova linha na tabela com `$('#tabela-body').append();`
4. **Persistência no LocalStorage (Exemplo 9):** Salvar a lista de registros com `localStorage.setItem()` e tratar a restauração dos dados ao abrir a página (F5).



Parte 3 — Desafio Extra: Filtro Dinâmico em Tempo Real

Desafio Extra – Filtro por Nome com jQuery

Como atividade de aprimoramento em sala:

1. Criar um campo de busca por nome acima da tabela (`$('#filtro-nome')`);
2. Implementar o evento `input` do jQuery para filtrar dinamicamente as linhas exibidas à medida que o usuário digita.

Checklist Final da Aula 2

Verificação Técnica Parte 2

Antes de finalizar o encontro, verifique se seu projeto atende aos critérios de aceitação:

O script do jQuery 3.7.1 está incluído e o arquivo `app.js` inicializa com `$(document).ready();`

Ao cadastrar um novo registro, a tabela é atualizada instantaneamente sem recarregar a página;

Ao pressionar F5 (Recarregar), os dados salvos no `LocalStorage` permanecem visíveis na tela.



4 Roteiro Prático 03 — Atividade Prática Autônoma: Calculadora de Pedidos & Carrinho de Compras

Objetivos

Objetivos da Atividade (Atividade Guiada de 2 Semanas — Calculadora de Pedidos)

Ao final desta atividade prática autônoma, o estudante será capaz de:

- Consolidação prática e aplicação integrada do **Bootstrap 5** e do **jQuery** em uma aplicação web funcional;
- Ler valores de campos de formulário (`<select>`, `<input type="number">`, `<input type="checkbox">`) e convertê-los com segurança para tipos numéricos utilizando `parseFloat()` e `parseInt()`;
- Tratar valores vazios ou não numéricos (NaN) utilizando o operador de fallback `|| 0`;
- Compreender a diferença e aplicar os eventos corretos do navegador: `.on('input')` para digitação em tempo real, `.on('change')` para alteração de seleção/switches e `.on('click')` para botões;
- Realizar o cálculo automático do subtotal, taxa de entrega, descontos por cupom e total geral em tempo real, sem necessidade de recarregar a página;
- Formatar valores monetários para o padrão brasileiro (R\$ X,XX) utilizando a função `.toFixed(2).replace('.', ',')`;
- Persistir e restaurar o rascunho do pedido no navegador utilizando o **LocalStorage** (`setItem`, `getItem`, `JSON.stringify`, `JSON.parse`);
- Organizar o projeto em arquivo externo `app.js` e publicar a aplicação no **GitHub Pages**.

Introdução e Contextualização da Atividade

Prezados estudantes da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Durante as próximas 2 semanas, enquanto o professor estará em afastamento de saúde, vocês realizarão uma atividade prática autônoma guiada para consolidar os conceitos aprendidos sobre **Bootstrap 5**, **jQuery**, **Cálculos Numéricos no Cliente** e **Persistência no LocalStorage**.

O tema proposto é o desenvolvimento da **Lanchonete Express — Calculadora de Pedidos & Carrinho de Compras**. A aplicação permitirá que o cliente escolha seu lanche principal, selecione adicionais opcionais (queijo, bacon, batata frita, bebida), ajuste a quantidade, escolha o tipo de entrega, aplique um



cupom de desconto e veja o **valor total a pagar atualizado instantaneamente na tela**.

Fundamentação Teórica: Como Ler e Calcular Valores de Campos com jQuery

Para realizar cálculos financeiros em uma página web, é necessário seguir três etapas indispensáveis:

4.2.1 1. Leitura e Conversão de Texto para Número (parseFloat e parseInt)

No HTML, todos os valores obtidos através de `$(seletor).val()` são retornados como **texto (string)**. Se tentarmos somar textos em JavaScript (ex: `"18.00" + "4.00"`), o resultado será uma concatenação (`"18.004.00"`), e não uma soma numérica.

Por isso, devemos converter o valor lido:

- `parseFloat(string)`: Converte o texto para número decimal (usado para preços);
- `parseInt(string)`: Converte o texto para número inteiro (usado para quantidades);
- **Tratamento de NaN (Not a Number)**: Caso o campo esteja vazio, o `parseFloat()` retornará NaN. Para evitar que os cálculos quebrem, usamos a instrução `|| 0`.

Exemplo de leitura segura com fallback zero:

```
const precoLanche = parseFloat($('#select-lanche').val()) || 0;  
const quantidade = parseInt($('#input-qtd').val()) || 1;
```

4.2.2 2. Soma de Múltiplos Checkboxes Marcados com .each()

Quando a aplicação possui múltiplos adicionais (ex: switches de queijo, bacon, batata frita), podemos percorrer apenas os checkboxes que o usuário marcou usando a combinação do seletor `:checked` com o método `.each()`:

Varredura de adicionais selecionados:

```
let somaAdicionais = 0;  
  
$('.check-adicional:checked').each(function() {  
    somaAdicionais += parseFloat($(this).val()) || 0;  
});
```



4.2.3 3. Guia de Eventos do Navegador (input, change e click)

Para proporcionar uma excelente experiência de usuário (UX), a calculadora deve reagir instantaneamente a qualquer alteração realizada na tela. Escolha o evento adequado para cada tipo de elemento:

Guia de Eventos do Navegador em jQuery

Evento jQuery	Componente Indicado e Comportamento
<code>.on('change')</code>	Indicado para select, checkbox e switch. Disparado quando o usuário altera a seleção de um dropdown ou marca/desmarca uma caixa.
<code>.on('input')</code>	Indicado para input type="number" ou text. Disparado instantaneamente a cada caractere digitado ou alterado no campo.
<code>.on('click')</code>	Indicado para botões. Disparado quando o usuário clica nos botões de incremento (+/-), aplicar cupom ou finalizar pedido.

4.2.4 4. Formatação Financeira em Real (R\$)

Para exibir valores numéricos formatados na moeda brasileira (R\$ 25,50), crie uma função auxiliar utilizando `.toFixed(2)` para fixar duas casas decimais e `.replace('.', ',')` para trocar o ponto pela vírgula:

Função de Formatação de Moeda Brasileira:

```
function formatarMoeda(valor) {  
    return 'R$ ' + valor.toFixed(2).replace('.', ',');  
}  
  
// Atualização da interface gráfica:  
$('#total-geral').text(formatarMoeda(totalCalculado));
```

Dicas de Estruturação Visual & Componentes do Bootstrap 5

Para que a aplicação tenha um visual profissional e limpo, recomendamos a utilização dos seguintes painéis e utilitários do **Bootstrap 5**:

1. **Grid de 2 Colunas** (`.row` e `.col-lg-7` / `.col-lg-5`): Divida a tela em dois painéis principais — a coluna da esquerda para o formulário de montagem do pedido e a coluna da direita para o resumo financeiro;
2. **Painel de Montagem** (`.card` e `.card-body`): Agrupe as opções do pedido em um cartão elevado com bordas suaves;



3. **Adicionais com Switches** (`.form-check .form-switch`): Utilize interruptores modernos para os adicionais opcionais (queijo, bacon, fritas), incluindo a etiqueta `.badge` com o preço;
4. **Input Groups para Quantidade** (`.input-group`): Combine os botões de decremento (-) e incremento (+) com o campo numérico no meio;
5. **Resumo Financeiro em Lista** (`.list-group .list-group-flush`): Exiba a discriminação do subtotal, taxa de entrega e desconto em uma lista estilizada;
6. **Destaque do Total Geral** (`.alert` ou **caixa personalizada**): Destaque o valor total a pagar com tamanho de fonte ampliado e fundo contrastante.

Roteiro Guiado Passo a Passo (Roteiro 3)

Siga os passos abaixo para construir a aplicação na sua pasta de exercícios (roteiro-3/exercicio-roteiro3/):

Passo 1: Estrutura HTML5 e Inclusão das CDNs

Crie o arquivo `index.html` e inclua as bibliotecas do **Bootstrap 5**, **Bootstrap Icons** e **jQuery 3.7.1** via CDN antes de fechar a tag `</body>`.

```
<!-- Bootstrap 5 CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<!-- Arquivo da sua lógica JS -->
<script src="app.js"></script>
```

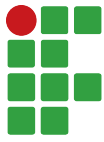
Passo 2: Formulário de Montagem do Pedido

Crie o menu de seleção do lanche principal atribuindo os preços no atributo `value` de cada `<option>`.

```
<select class="form-select" id="select-lanche">
  <option value="0">-- Selecione um lanche --</option>
  <option value="18.00">X-Burger Clássico - R$ 18,00</option>
  <option value="22.00">X-Salada Especial - R$ 22,00</option>
  <option value="26.00">X-Bacon Supremo - R$ 26,00</option>
</select>
```

Passo 3: Lógica de Cálculo no `app.js`

No arquivo `app.js`, estruture a função `$(document).ready()` e crie a função principal `calcularTotal()`:



```
$(document).ready(function() {

    function calcularTotal() {
        const precoLanche = parseFloat($('#select-lanche').val()) || 0;

        let somaAdicionais = 0;
        $('.check-adicional:checked').each(function() {
            somaAdicionais += parseFloat($(this).val()) || 0;
        });

        let qtd = parseInt($('#input-qtd').val()) || 1;
        const subtotal = (precoLanche + somaAdicionais) * qtd;
        const taxaEntrega = parseFloat($('#select-entrega').val()) || 0;

        const totalGeral = subtotal + taxaEntrega;

        $('#total-geral').text('R$ ' + totalGeral.toFixed(2).replace('.', ','));
    }

    // Escutadores em tempo real
    $('#select-lanche, #select-entrega, .check-adicional').on('change', calcularTotal);
    $('#input-qtd').on('input change', calcularTotal);

});
```

Passo 4: Persistência no LocalStorage

Implemente o salvamento automático do rascunho do pedido no localStorage ao clicar em "Finalizar Pedido" e a restauração automática ao pressionar F5:

```
// Salvar no LocalStorage
$('#btn-finalizar').on('click', function() {
    const pedido = {
        lanche: $('#select-lanche').val(),
        qtd: $('#input-qtd').val(),
        entrega: $('#select-entrega').val()
    };
    localStorage.setItem('rascunho_pedido', JSON.stringify(pedido));
    alert('Pedido salvo no navegador!');
});
```



Checklist de Conclusão e Orientações de Entrega

Antes de enviar sua atividade, verifique se você concluiu todos os itens da lista abaixo:

- ☐ A página carrega corretamente sem erros no console do navegador (F12);
- ☐ O layout é responsivo e utiliza os cartões e formulários do Bootstrap 5;
- ☐ O cálculo do valor total reage em tempo real a qualquer mudança nos selects, checkboxes ou quantidade;
- ☐ Os valores numéricos são devidamente formatados em Reais (R\$ X,XX);
- ☐ O rascunho do pedido é salvo e restaurado via `localStorage`;
- ☐ O código foi commitado no repositório Git pessoal e publicado via **GitHub Pages**.



5 Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas

Objetivos

Objetivos da Atividade (Módulo 2 — ES6 Modules & Padrão em Camadas)

Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Superar a arquitetura monolítica de arquivo único (`app.js`), organizando o frontend em múltiplos módulos especializados utilizando **ES6 Modules** (`import` e `export`);
- Declarar o ponto de entrada modular na página web através da diretiva:
`<script type="module" src="js/main.js"></script>`;
- Compreender e aplicar a separação estrita de responsabilidades no cliente em três camadas:
 - **Camada de Serviços (`services/`):** Regras de negócio, cálculos e persistência pura no `localStorage` sem acoplamento ao DOM;
 - **Camada de Visão (`views/`):** Renderização de cards, tabelas, modais e Toasts via *template literals* do Bootstrap 5;
 - **Camada Utilitária (`utils/`):** Funções puras reutilizáveis de formatação (moeda brasileira, telefone, máscaras);
 - **Orquestrador Central (`main.js`):** Captura de eventos do usuário e coordenação entre camadas.
- Implementar o **CRUD Completo** (*Create, Read, Update, Delete*) com persistência em armazenamento local;
- Implementar **validação visual no cliente** com feedback imediato ao usuário utilizando as classes nativas do Bootstrap 5 (`.is-valid`, `.is-invalid` e `.invalid-feedback`);
- Disparar notificações flutuantes dinâmicas (**Bootstrap Toasts**) para confirmar operações de cadastro, alteração e exclusão;
- Implementar filtragem reativa em tempo real por categoria de serviço na vitrine comunitária.

Introdução e Contextualização do Projeto

Nos Roteiros 1, 2 e 3, trabalhamos com scripts JavaScript concentrados em um único arquivo (`app.js`). Embora essa abordagem seja excelente para aprender os fundamentos, ela se torna inviável em aplicações



reais de médio e grande porte, onde a mistura de seletores do DOM, regras de validação e persistência gera o chamado *"código espaguete"*.

No **Roteiro Prático 04**, damos o passo definitivo rumo à arquitetura profissional de software no frontend. Desenvolveremos a **Vitrine Comunitária de Empreendedores Locais de Garopaba**, uma aplicação que permite cadastrar, listar, filtrar, editar e gerenciar serviços da comunidade local.

Este projeto serve também como a **base tecnológica inicial para a Ação Curricularizada de Extensão na SNCT (Semana Nacional de Ciência e Tecnologia)**.

Estrutura Arquitetural de Pastas

O projeto deve ser organizado na pasta `roteiro-4/exercicio-roteiro4/` seguindo rigorosamente a estrutura modular:

Árvore de Diretórios Modular do Projeto:

```
roteiro-4/exercicio-roteiro4/
|-- index.html      <-- Vitrine e forms (BS5)
|-- styles.css      <-- Estilos e transições
`-- js/
    |-- main.js      <-- Orquestrador central
    |-- services/
    |   `-- vitrineService.js <-- CRUD Storage
    |-- views/
    |   `-- vitrineView.js    <-- HTML dinâmico
    `-- utils/
        `-- formatters.js    <-- Formatações
```

Fundamentação Teórica: Como Funcionam os ES6 Modules

5.3.1 1. Declaração no HTML (type="module")

Ao declarar `<script type="module" src="js/main.js"></script>`, o navegador aplica comportamentos modernos essenciais:

- **Modo Estrito Automático ("use strict"):** Impede o uso de variáveis não declaradas e falhas silenciosas;
- **Escopo Fechado de Módulo:** Variáveis e funções criadas dentro de um arquivo não poluem o escopo global (window);
- **Carregamento Assíncrono com Adiamiento (defer):** O script aguarda o parsing completo do DOM antes de executar.



5.3.2 2. Exportação e Importação (export e import)

Para compartilhar recursos entre arquivos, usamos a sintaxe explícita de módulos:

Exemplo de Exportação Nomeada e Importação:

```
// No arquivo js/utils/formatters.js
export function formatarTelefone(telefone) {
  return telefone.replace(
    /^(\d{2})(\d{5})(\d{4})/,
    '($1) $2-$3'
  );
}

// No arquivo js/main.js
import { formatarTelefone }
  from './utils/formatters.js';

// Resultado: (48) 99999-8888
console.log(formatarTelefone('48999998888'));
```

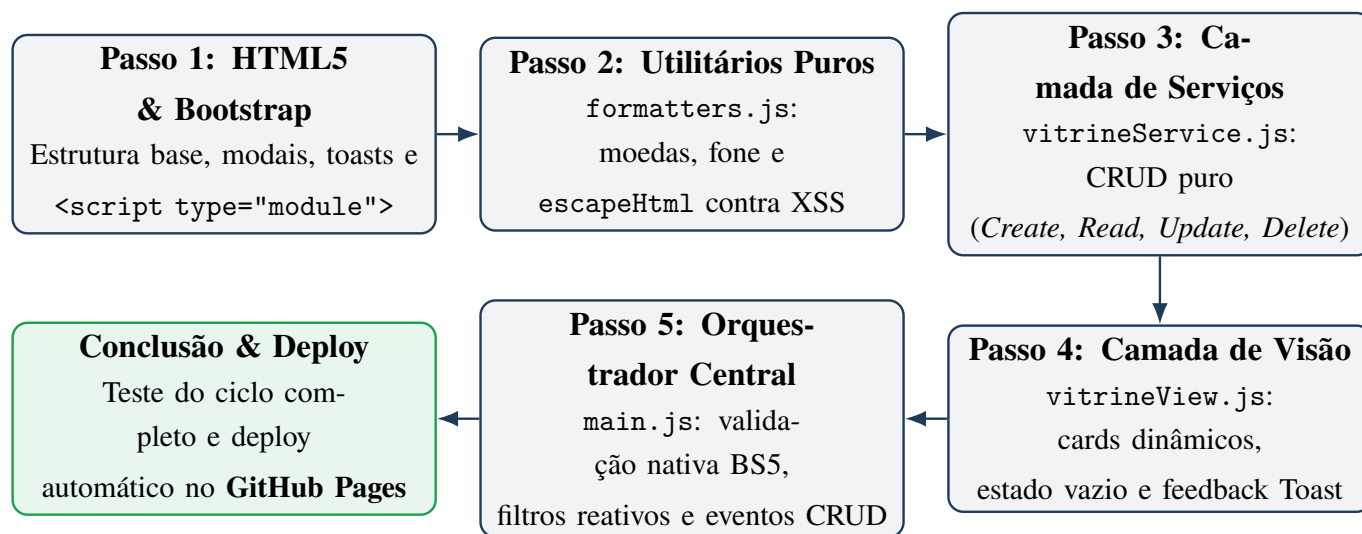


Figura 2: Pipeline arquitetural e sequência cronológica de implementação do Roteiro 04.



Passo a Passo Guiado do Roteiro

Passo 1.1: Estrutura HTML5 Base e Ponto de Entrada (index.html)

Crie o arquivo `index.html` com as CDNs do Bootstrap 5, o ponto de montagem (`#vitrineContainer`) e a tag `<script type="module">`:

```
<!-- index.html (Parte 1: Estrutura Base e Scripts) -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Vitrine de Garopaba - Roteiro 04</title>
  <!-- Bootstrap 5 CSS & Bootstrap Icons CDN -->
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/
      bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/
      bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <!-- Container de Toasts e Barra de Filtros -->
  <div id="toastNotificacao"
    class="toast position-fixed top-0 end-0 m-3">...</div>
  <select class="form-select" id="filtroCategoria">
    <option value="todas">Todas as Categorias</option>
  </select>
  <button class="btn btn-warning" data-bs-toggle="modal"
    data-bs-target="#modalCadastro">Divulgar</button>

  <!-- Ponto de Montagem dos Cards Dinâmicos -->
  <div class="row" id="vitrineContainer"></div>

  <!-- Bootstrap Bundle JS + Entrada Modular ES6 -->
  <script src="https://cdn.jsdelivr.net/npm/
    bootstrap@5.3.3/dist/js/
      bootstrap.bundle.min.js"></script>
  <script type="module" src="js/main.js"></script>
</body>
</html>
```



Passo 1.2: Modal de Cadastro e Edição (index.html)

Estruture o modal Bootstrap 5 contendo o formulário com validações nativas e o campo oculto #servicoId:

```
<!-- index.html (Parte 2: Modal de Cadastro/Edição) -->
<div class="modal fade" id="modalCadastro" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <form id="formCadastro" novalidate>
        <!-- Campo oculto para ID na operação Update -->
        <input type="hidden" id="servicoId" value="">

        <input type="text" id="nome"
          class="form-control" required>
        <select id="categoria" class="form-select" required>
          <option value="">Selecione...</option>
        </select>
        <input type="text" id="bairro"
          class="form-control" required>
        <input type="number" id="precoBase"
          class="form-control" required>
        <input type="tel" id="telefone"
          class="form-control" required>
        <textarea id="descricao"
          class="form-control" required></textarea>

        <button type="submit" id="btnSalvarModal"
          class="btn btn-success">
          <span id="btnSalvarTexto">Salvar na Vitrine</span>
        </button>
      </form>
    </div>
  </div>
</div>
```

**Passo 2: Estruturação dos Utilitários Puros (js/utils/formatters.js)**

Crie o arquivo de funções utilitárias que realizam operações puras de transformação de dados:

```
// js/utils/formatters.js
export function formatarMoeda(valor) {
  const num = parseFloat(valor) || 0;
  return num.toLocaleString('pt-BR', {
    style: 'currency',
    currency: 'BRL'
  });
}

export function formatarTelefone(fone) {
  const digits = (fone || '').replace(/\D/g, '');
  if (digits.length === 11) {
    return digits.replace(
      /^(\d{2})(\d{5})(\d{4})/,
      '($1) $2-$3'
    );
  }
  return fone;
}

export function escapeHtml(texto) {
  const div = document.createElement('div');
  div.textContent = texto;
  return div.innerHTML;
}
```



Passo 3: Camada de Dados e Serviços (js/services/vitrineService.js)

Esta camada gerencia o ciclo de vida dos dados no localStorage sem qualquer manipulação de elementos HTML (operações CRUD puras):

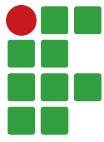
```
// js/services/vitrineService.js
const STORAGE_KEY = 'garopaba_vitrine_servicos';

export function obterServicos() {
  const dados = localStorage.getItem(STORAGE_KEY);
  return dados ? JSON.parse(dados) : [];
}

export function salvarServico(novo) {
  const servicos = obterServicos();
  const completo = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novo
  };
  servicos.unshift(completo);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return completo;
}

export function atualizarServico(id, dados) {
  const servicos = obterServicos();
  const idx = servicos.findIndex(s => s.id === id);
  if (idx !== -1) {
    servicos[idx] = { ...servicos[idx], ...dados,
      dataEdicao: new Date().toISOString() };
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return servicos[idx];
  }
  return null;
}

export function removerServico(id) {
  const servicos = obterServicos().filter(s => s.id !== id);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicos;
}
```



Passo 4.1: Template do Card de Serviço (js/views/vitrineView.js)

Gera a estrutura HTML de cada card individual com dados formatados e botões de ação:

```
// js/views/vitrineView.js (Parte 1: Template do Card)
import { formatarMoeda, formatarTelefone, escapeHtml }
  from '../utils/formatters.js';

export function criarCardHtml(s) {
  const tel = (s.telefone || '').replace(/\D/g, '');
  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card">
        <div class="card-body">
          <div class="d-flex justify-content-between mb-2">
            <span class="badge bg-success">
              ${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-primary">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>
          <h5 class="card-title fw-bold mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>
            ${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small">
            ${escapeHtml(s.descricao)}
          </p>
        </div>
        <div class="card-footer bg-transparent border-0 pt-0 pb-3
          d-flex justify-content-between align-items-center">
          <a href="https://wa.me/55${tel}" target="_blank"
            class="btn btn-sm btn-outline-success">
            <i class="bi bi-whatsapp me-1"></i>
            ${formatarTelefone(s.telefone)}
          </a>
          <div class="d-flex gap-1">
            <button class="btn btn-sm btn-outline-primary btn-editar"
              data-id="${s.id}" title="Editar">
              <i class="bi bi-pencil-square"></i>
            </button>
            <button class="btn btn-sm btn-outline-danger btn-excluir"
              data-id="${s.id}" title="Excluir">
              <i class="bi bi-trash"></i>
            </button>
          </div>
        </div>
      </div>
    </div>`;
}
```



Passo 4.2: Renderização da Vitrine e Estado Vazio

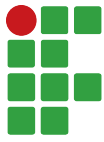
Itera sobre a coleção de serviços chamando a função de template e tratando o estado vazio:

```
// js/views/vitrineView.js (Parte 2: Renderização)
export function renderizarCards(servicos, container) {
  if (!servicos || servicos.length === 0) {
    container.innerHTML = `
      <div class="col-12 text-center py-5">
        <i class="bi bi-inbox fs-1 text-muted"></i>
        <h5 class="text-muted mt-2">
          Nenhum serviço cadastrado nesta categoria.
        </h5>
      </div>`;
    return;
  }
  container.innerHTML = servicos.map(criarCardHtml).join('');
}
```

Passo 4.3: Notificações Flutuantes (Bootstrap Toasts)

Dispara notificações assíncronas flutuantes através da API nativa do Bootstrap Toast:

```
// js/views/vitrineView.js (Parte 3: Toasts)
export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastBody = document.getElementById('toastMensagem');
  if (toastEl && toastBody) {
    toastBody.innerText = mensagem;
    toastEl.className = `toast align-items-center text-bg-${tipo} border-0`;
    const toast = bootstrap.Toast.getOrCreateInstance(toastEl);
    toast.show();
  }
}
```



Passo 5.1: Inicialização e Reatividade (js/main.js)

Configura as referências aos elementos do DOM e implementa a renderização reativa:

```
// js/main.js (Parte 1: Inicialização e Leitura)
import {
  obterServicos, salvarServico,
  atualizarServico, removerServico
} from './services/vitrineService.js';
import {
  renderizarCards, exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const modalEl = document.getElementById('modalCadastro');
  const servicoIdInput = document.getElementById('servicoId');

  function atualizarInterface() {
    const todos = obterServicos();
    const filtro = filtroCategoria.value;
    const filtrados = filtro === 'todas'
      ? todos
      : todos.filter(s => s.categoria === filtro);
    renderizarCards(filtrados, container);
    if (contadorEl) {
      contadorEl.innerText = `${todos.length} cadastrados`;
    }
  }

  filtroCategoria.addEventListener('change', atualizarInterface);
  atualizarInterface();
});
```



Passo 5.2: Validação e Submissão — Create & Update (js/main.js)

Gerencia a validação nativa do formulário e alterna entre novo cadastro e atualização:

```
// js/main.js (Parte 2: Submissão do Formulário)
form.addEventListener('submit', (e) => {
  e.preventDefault();
  if (!form.checkValidity()) {
    e.stopPropagation();
    form.classList.add('was-validated');
    return;
  }

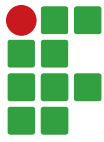
  const idAtual = servicoIdInput ? servicoIdInput.value : '';
  const getVal = (id) => document.getElementById(id).value.trim();

  const dados = {
    nome: getVal('nome'),
    categoria: document.getElementById('categoria').value,
    bairro: getVal('bairro'),
    precoBase: parseFloat(getVal('precoBase')) || 0,
    telefone: getVal('telefone'),
    descricao: getVal('descricao')
  };

  if (idAtual) {
    atualizarServico(idAtual, dados);
    exibirToast('Serviço atualizado com sucesso!');
  } else {
    salvarServico(dados);
    exibirToast('Empreendimento cadastrado com sucesso!');
  }

  form.reset();
  if (servicoIdInput) servicoIdInput.value = '';
  form.classList.remove('was-validated');

  if (modalEl) {
    const modal = bootstrap.Modal.getInstance(modalEl);
    if (modal) modal.hide();
  }
  atualizarInterface();
});
```



Passo 5.3: Delegação de Eventos — Edit & Delete (js/main.js)

Escuta cliques nos botões dos cards para carregar dados no modal ou remover itens:

```
// js/main.js (Parte 3: Delegação de Cliques nos Cards)
container.addEventListener('click', (e) => {
  // 1. Operação Update (Edição)
  const btnEditar = e.target.closest('.btn-editar');
  if (btnEditar) {
    const id = btnEditar.getAttribute('data-id');
    const item = obterServicos().find(s => s.id === id);
    if (item) {
      if (servicoIdInput) servicoIdInput.value = item.id;
      document.getElementById('nome').value = item.nome;
      document.getElementById('categoria').value = item.categoria;
      document.getElementById('bairro').value = item.bairro;
      document.getElementById('precoBase').value = item.precoBase;
      document.getElementById('telefone').value = item.telefone;
      document.getElementById('descricao').value = item.descricao;

      if (modalEl) {
        const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
        modal.show();
      }
    }
    return;
  }

  // 2. Operação Delete (Exclusão)
  const btnExcluir = e.target.closest('.btn-excluir');
  if (btnExcluir) {
    const id = btnExcluir.getAttribute('data-id');
    if (confirm('Deseja realmente remover este serviço da vitrine?')) {
      removerServico(id);
      atualizarInterface();
      exibirToast('Serviço removido da vitrine.', 'warning');
    }
  }
});
}); // Fim do DOMContentLoaded
```



Checklist de Conclusão e Critérios Avaliativos

Antes de realizar o commit final no repositório, certifique-se de validar todos os itens:

- ☐ O projeto utiliza estritamente o padrão ES6 Modules com a tag:
`<script type="module" src="js/main.js"></script>`;
- ☐ A estrutura em pastas (js/services/, js/views/, js/utils/) foi rigorosamente respeitada;
- ☐ O arquivo vitrineService.js opera puramente sobre o localStorage sem manipular elementos do DOM;
- ☐ As quatro operações do CRUD (**Create, Read, Update e Delete**) foram implementadas e testadas com persistência no navegador;
- ☐ O formulário exibe validações visuais nativas do Bootstrap 5 (.is-invalid) quando campos obrigatórios não são preenchidos;
- ☐ A vitrine reage instantaneamente aos filtros de categoria sem recarregar a página;
- ☐ O código está devidamente publicado e em pleno funcionamento no **GitHub Pages**.